

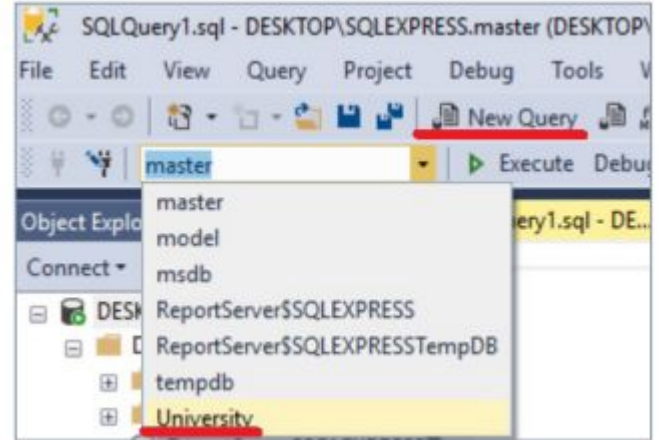
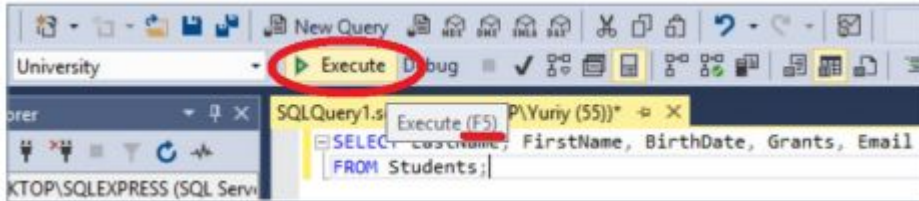
Запросы

Модуль 2



Создание запроса

1. Кнопка New Query на панели инструментов
2. Выбрать необходимую базу данных
3. Кнопка Execute для запуска



SELECT, FROM

SELECT + имена столбцов - позволяет считывать информацию из БД

FROM + названия таблиц - обязательный оператор после SELECT, источник получения данных

```
SELECT columnName1, columnName2, ...
```

```
FROM tableName;
```

Пример University

```
SELECT LastName, FirstName, BirthDate, Grants, Email  
FROM Students;
```

	LastName	FirstName	BirthDate	Grants	Email
1	Doe	John	1998-02-11	NULL	jh@net.eu
2	Jones	Jack	1997-11-05	1256.00	jj@net.eu
3	Johnson	Joshua	1997-05-23	NULL	jo@net.eu

Результат тот же, но выполняется дольше:

```
SELECT *  
FROM Students;
```

Формирование новых столбцов

```
SELECT FirstName + ' ' + LastName AS FullName, BirthDate, Grants, Email  
FROM Students;
```

Оператор AS - задаёт псевдоним

	FullName	BirthDate	Grants	Email
1	John Doe	1998-02-11	NULL	jh@net.eu
2	Jack Jones	1997-11-05	1256.00	jj@net.eu
3	Joshua Johnson	1997-05-23	NULL	jo@net.eu

Арифметические действия

«Как изменятся стипендии студентов, если их увеличить на 20 процентов?»

```
SELECT FirstName + ' ' + LastName AS FullName, Grants*1.2 AS [Plus 20 percent]
```

```
FROM Students;
```

	FullName	Plus 20 percent
1	John Doe	NULL
2	Jack Jones	1507.200
3	Joshua Johnson	NULL

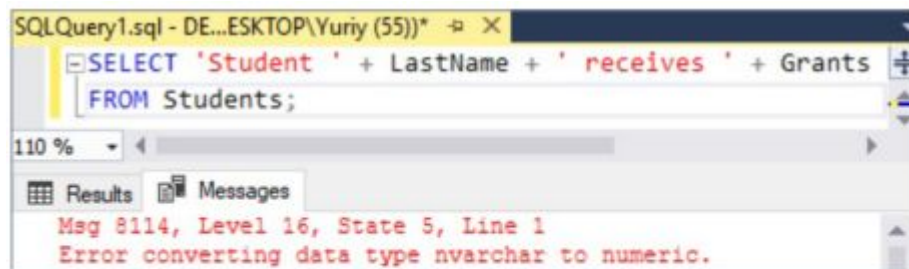


Рисунок 1.6. Ошибка: недопустимое приведение типов данных

CAST() и CONVERT()

На прошлом слайде ошибка произошла из-за невозможности преобразовать вещественные значения столбца Grants к строковому типу данных.

CAST() и **CONVERT()** - приведение данных к требуемому типу.

```
SELECT 'Student ' + LastName + ' receives ' + CAST(Grants AS nvarchar(10))
```

```
FROM Students;
```

```
SELECT 'Student ' + LastName + ' receives ' + CONVERT(nvarchar(10), Grants)
```

```
FROM Students;
```

	(No column name)
1	NULL
2	Student Jones receives 1256.00
3	NULL

TOP

Оператор **TOP** - позволяет получить первые записи либо в количественном, либо процентном отношении.

```
SELECT TOP 2 LastName, FirstName, BirthDate
```

```
FROM Students;
```

	LastName	FirstName	BirthDate
1	Doe	John	1998-02-11
2	Jones	Jack	1997-11-05

```
SELECT TOP 20 PERCENT LastName, FirstName, BirthDate
```

```
FROM Students;
```

	LastName	FirstName	BirthDate
1	Doe	John	1998-02-11

PERCENT - ключевое слово

DISTINCT

Нам надо определить, какие имена у студентов в «нашем учебном заведении», при этом следует исключить повторения.

DISTINCT - исключение повторяющихся значений

```
SELECT DISTINCT FirstName  
FROM Students;
```

	FirstName
1	Jack
2	John
3	Joshua

WHERE

WHERE + требуемые условия -

позволяет отфильтровать получаемые данные, задав определенные условия.

```
SELECT columnName1, columnName2, ...
```

```
FROM tableName
```

```
WHERE condition;
```

Операторы сравнения:

- = — равно;
- <> — не равно;
- > — больше;
- >= — больше или равно;
- !> — не больше чем;
- < — меньше;
- <= — меньше или равно;
- !< — не меньше чем.

WHERE

- Необходимо вывести стипендию и фамилии всех студентов, у которых размер стипендии превышает 1000.

```
SELECT LastName, Grants
```

```
FROM Students
```

```
WHERE Grants > 1000;
```

	LastName	Grants
1	Jones	1256.00

- Необходимо получить информацию о студентах, в имени которых не больше чем четыре символа
- **LEN()** - длина строки

```
SELECT Id, LastName, FirstName,  
BirthDate, Grants, Email
```

```
FROM Students
```

```
WHERE LEN(FirstName) !> 4
```

	Id	LastName	FirstName	BirthDate	Grants	Email
1	1	Doe	John	1998-02-11	NULL	jh@net.eu
2	2	Jones	Jack	1997-11-05	1256.00	jj@net.eu

AND

AND - объединяет два условия и возвращает истину, если они оба верны

Требуется получить информацию о студентах, которые родились осенью

```
SELECT LastName, FirstName, BirthDate
```

```
FROM Students
```

```
WHERE MONTH(BirthDate) >= 9 AND MONTH(BirthDate) <= 11;
```

MONTH() - позволяет получить номер месяца из переданной даты

DAY(), YEAR() - получить день и год соответственно

	LastName	FirstName	BirthDate
1	Jones	Jack	1997-11-05

OR

OR - объединяет два условия и возвращает истину, если хотя бы одно из условий верно.

Необходимо отобразить информацию о студентах, у которых либо четный год рождения, либо нечетный день рождения

```
SELECT LastName, FirstName, BirthDate
```

```
FROM Students
```

```
WHERE YEAR(BirthDate) % 2 = 0 OR DAY(BirthDate) % 2 <> 0;
```

% - операция деления по модулю

	LastName	FirstName	BirthDate
1	Doe	John	1998-02-11
2	Jones	Jack	1997-11-05
3	Johnson	Joshua	1997-05-23

IS NULL

IS NULL - позволяет проверить значения столбцов на неопределенность (значение NULL)

Получить список всех студентов, которые не получают стипендию.

```
SELECT LastName, FirstName, Grants  
FROM Students  
WHERE Grants IS NULL;
```

	LastName	FirstName	Grants
1	Doe	John	NULL
2	Johnson	Joshua	NULL

NOT

NOT - позволяет задать противоположное условие.

Требуется получить информацию о студентах, фамилии которых указывались при заполнении таблицы Students, то есть не равны Doe.

```
SELECT Id, LastName, FirstName, BirthDate,  
Grants, Email
```

```
FROM Students
```

```
WHERE NOT LastName = 'Doe';
```

	Id	LastName	FirstName	BirthDate	Grants	Email
1	2	Jones	Jack	1997-11-05	1256.00	jj@net.eu
2	7	Johnson	Joshua	1997-05-23	NULL	jo@net.eu

Необходима информация обо всех студентах, которые получают стипендию

```
SELECT LastName, FirstName,  
Grants
```

```
FROM Students
```

```
WHERE Grants IS NOT NULL;
```

	LastName	FirstName	Grants
1	Jones	Jack	1256.00

ORDER BY

ORDER BY - позволяет упорядочить полученную информацию

```
SELECT columnName1, columnName2, ...
```

```
FROM tableName
```

```
ORDER BY columnName1 ASC | DESC, ...;
```

Можно указать **количество столбцов**, по которым производится сортировка и **направление** этой сортировки

DESC (от descending) - ключевое слово для сортировки по убыванию

ASC (от ascending) - сортировка по возрастанию. Можно не указывать.

*Список студентов,
отсортированный по дате
рождения*

```
SELECT LastName, FirstName,  
BirthDate
```

```
FROM Students
```

```
ORDER BY BirthDate;
```

	LastName	FirstName	BirthDate
1	Johnson	Joshua	1997-05-23
2	Jones	Jack	1997-11-05
3	Doe	John	1998-02-11

*Список студентов надо
отсортировать как по убыванию
(столбец LastName), так и по
возрастанию (столбец FirstName)*

```
SELECT LastName, FirstName, BirthDate
```

```
FROM Students
```

```
ORDER BY LastName DESC, FirstName  
ASC;
```

	LastName	FirstName	BirthDate
1	Jones	Jack	1997-11-05
2	Johnson	Joshua	1997-05-23
3	Doe	John	1998-02-11

IN

	LastName	FirstName	BirthDate	Grants	Email
1	Doe	John	1998-02-11	NULL	jh@net.eu
2	Jones	Jack	1997-11-05	1256.00	jj@net.eu

Необходимо получить информацию о студентах, которых зовут John, David или Jack

```
SELECT LastName, FirstName, BirthDate, Grants, Email
```

```
FROM Students
```

```
WHERE FirstName='John' OR FirstName='David' OR FirstName='Jack';
```

IN - обеспечивает сравнение значения указанного столбца с любым значением из указанного множества

```
SELECT LastName, FirstName, BirthDate, Grants, Email
```

```
FROM Students
```

```
WHERE FirstName IN ('John', 'David', 'Jack');
```

BETWEEN

BETWEEN - проверка на попадание значения определенного столбца в некий диапазон

Ключевое слово BETWEEN прописывается *после имени столбца*, а после него необходимо указать *нижнюю и верхнюю границу* диапазона, разделенных оператором **AND**, границы диапазона также *учитываются*.

Необходимо определить студентов, которые родились в 1997 году

```
SELECT LastName, FirstName, BirthDate
```

```
FROM Students
```

```
WHERE BirthDate BETWEEN '1997-01-01'
```

```
AND '1997-12-31';
```

	LastName	FirstName	BirthDate
1	Jones	Jack	1997-11-05
2	Johnson	Joshua	1997-05-23

Сравнение строковых значений

Необходимо вывести всех студентов, фамилии которых не начинаются на буквы E, F и G

```
SELECT LastName, FirstName
```

```
FROM Students
```

```
WHERE LastName NOT BETWEEN 'E' AND 'G';
```

Сравнение будет осуществляться в соответствии с кодами символов

	LastName	FirstName
1	Doe	John
2	Jones	Jack
3	Johnson	Joshua

LIKE - использование шаблона

Получить всех студентов, фамилия которых начинается на букву J или M, а имя заканчивается на букву k

```
SELECT LastName, FirstName,  
BirthDate, Grants, Email
```

```
FROM Students
```

```
WHERE LastName LIKE 'J[M]%' AND  
FirstName LIKE '%k';
```

	LastName	FirstName	BirthDate	Grants	Email
1	Jones	Jack	1997-11-05	1256.00	jj@net.eu

Служебные символы:

- % — соответствует любой последовательности символов от 0 и более;
- _ — представляет любой одиночный символ;
- [] — задает последовательность или диапазон возможных символов;
- [^] — задает последовательность или диапазон символов, которые должны отсутствовать.

Студенты, у которых в адресе электронной почты вторая буква не совпадает ни с одной букв из диапазона от f до m

```
SELECT LastName, FirstName, BirthDate, Grants, Email
```

```
FROM Students
```

```
WHERE Email LIKE '_[^f-m]%';
```

	LastName	FirstName	BirthDate	Grants	Email
1	Johnson.	Joshua	1997-05-23	NULL	jo@net.eu

INSERT

INSERT - заполнение таблиц программным путем

1 способ:

```
INSERT INTO tableName (columnName1, columnName2, ...)
```

```
VALUES (value1, value2, ...);
```

- Необходимо указывать, как минимум, все столбцы таблицы, которые не могут принимать неопределенное (NULL) значение
- Не обязательно соблюдать правильный порядок следования имен столбцов в таблице

1 способ

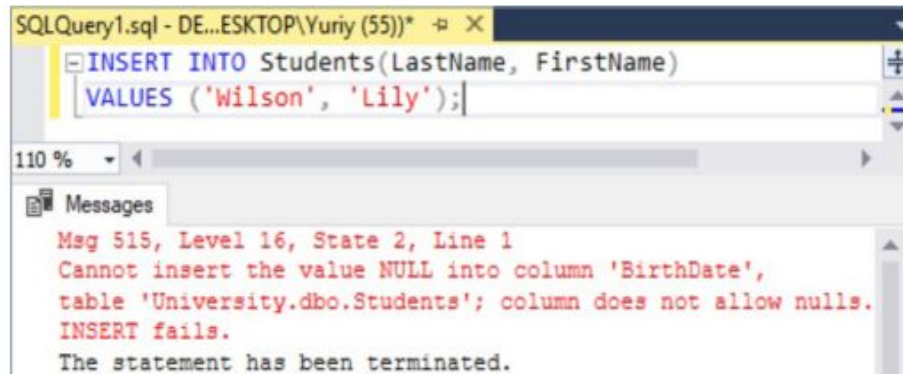
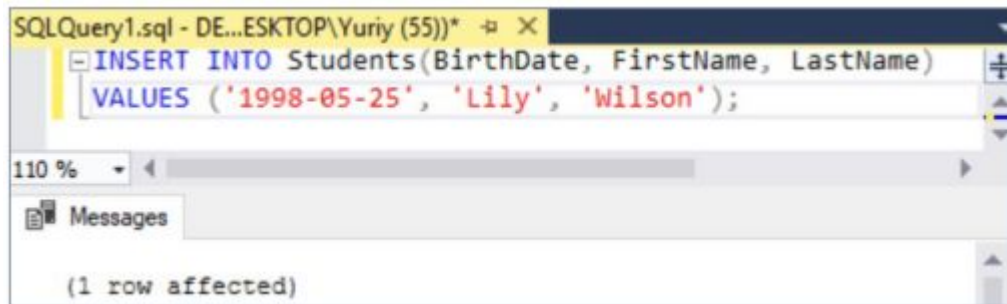


Рисунок 3.1. Ошибка: невозможно вставить NULL-значение в столбец BirthDate



Проверка:

SELECT LastName, FirstName,
BirthDate, Grants, Email

FROM Students;

	LastName	FirstName	BirthDate	Grants	Email
1	Doe	John	1998-02-11	NULL	jh@net.eu
2	Jones	Jack	1997-11-05	1256.00	jj@net.eu
3	Johnson	Joshua	1997-05-23	NULL	jo@net.eu
4	Wilson	Lily	1998-05-25	NULL	NULL

2 способ

- Названия столбцов таблицы не указываются

```
INSERT INTO tableName
```

```
VALUES (value1, value2, ...);
```

- Необходимо указывать значения для всех столбцов, которые не могут принимать неопределенное значение, обязательно соблюдая порядок их следования в таблице

```
INSERT INTO Students
```

```
VALUES ('Evans', 'Grace', '1998-08-30', NULL, 'eg@net.eu');
```

INSERT INTO SELECT

Заполнение существующей таблицы значениями, полученными в результате выполнения SQL-запроса

1 способ - копируем все данные из таблицы, являющейся источником данных

```
INSERT INTO destinationTable
```

```
SELECT * FROM sourceTable
```

```
WHERE condition;
```

2 способ - явное указание, как названий столбцов таблицы-источника, подлежащих копированию, так и названий столбцов таблицы-приемника данных

```
INSERT INTO destinationTable (columnName1,  
columnName2, ...)
```

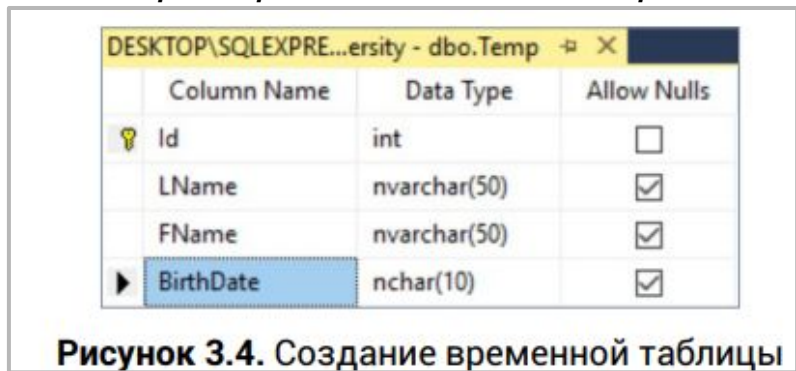
```
SELECT columnName1, columnName2, ...
```

```
FROM sourceTable
```

```
WHERE condition;
```

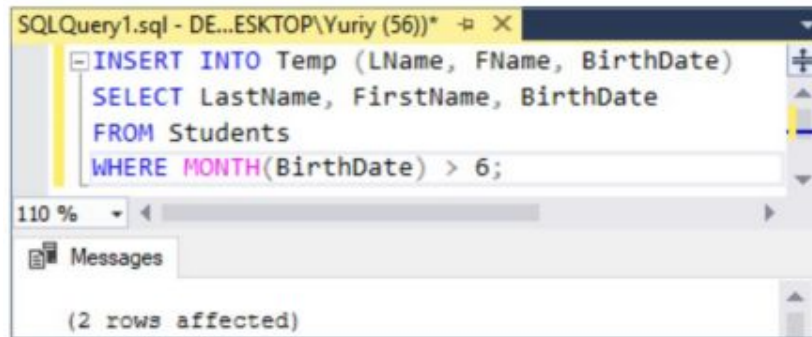
Создадим временную таблицу

Заполним ее только теми сведениями о студентах (из таблицы Students), которые родились во второй половине года



Проверка:

```
SELECT LName, FName, BirthDate  
FROM Temp;
```



	LName	FName	BirthDate
1	Jones	Jack	1997-11-05
2	Evans	Grace	1998-08-30

Рисунок 3.6. Проверка правильности добавления данных

SELECT INTO

Создание новой таблицы на основе существующей, путем копирования значений всех указанных столбцов

SELECT columnName1, columnName2, ...

INTO newTable

FROM existingTable

WHERE condition;

Создадим в нашей базе данных *University* новую таблицу с информацией о студентах, электронный адрес которых известен.

Проверка:

SELECT *

FROM Temp1;

	LastName	FirstName	BirthDate	Email
1	Doe	John	1998-02-11	jh@net.eu
2	Jones	Jack	1997-11-05	jj@net.eu
3	Johnson	Joshua	1997-05-23	jo@net.eu
4	Evans	Grace	1998-08-30	eg@net.eu

Рисунок 3.8. Проверка информации, записанной в новую таблицу

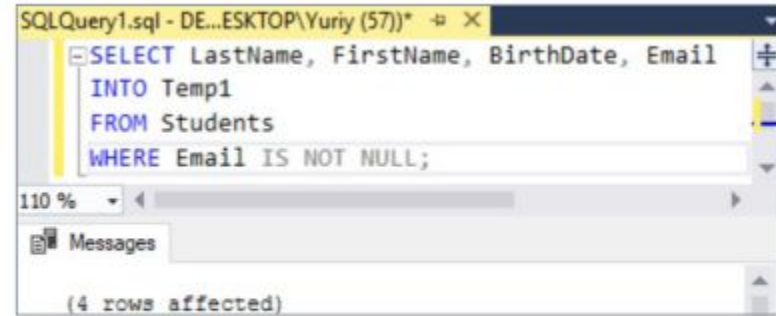


Рисунок 3.7. Создание новой таблицы на основе существующей

UPDATE

UPDATE - позволяет изменить любую запись

UPDATE tableName

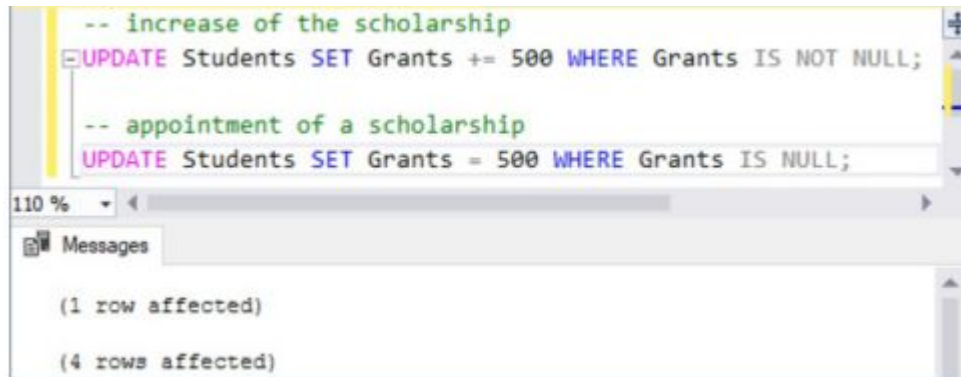
SET columnName1 = value1, columnName2 = value2, .

WHERE condition;

Если не указать условие (WHERE), то изменятся значения в соответствующем столбце по всей таблице

Необходимо выплачивать стипендию всем студентам. Тем, кто её уже получал - стипендию стоит повысить.

- Обратите внимание на порядок выполнения запросов!



```
-- increase of the scholarship
UPDATE Students SET Grants += 500 WHERE Grants IS NOT NULL;

-- appointment of a scholarship
UPDATE Students SET Grants = 500 WHERE Grants IS NULL;
```

110 %

Messages

(1 row affected)

(4 rows affected)

Проверка:

SELECT LastName, FirstName, Grants

FROM Students;

	LastName	FirstName	Grants
1	Doe	John	500.00
2	Jones	Jack	1756.00
3	Johnson	Joshua	500.00
4	Wilson	Lily	500.00
5	Evans	Grace	500.00

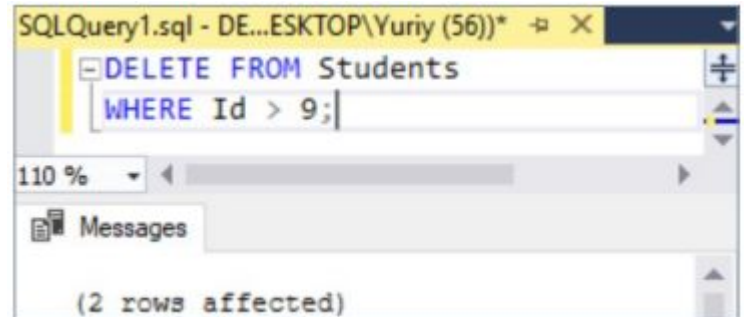
DELETE

DELETE - позволяет удалить определенную информацию из таблиц базы данных программным путем

DELETE FROM tableName

WHERE condition;

Необходимо удалить все записи из таблицы Students, у которых уникальный идентификатор (Id) больше 9



Транзакции

Транзакция - последовательность действий, выполняемых как единое целое с возможностью отмены каждого из них в случае ошибки

ACID:

- **Atomicity (атомарность)** — целостность транзакции, то есть для успешного завершения транзакции должны выполняться либо все операции, либо ни одна из них
- **Consistency (согласованность)** — целостность информации независимо от того, успешно завершилась транзакция или нет
- **Isolation (изолированность)** — все транзакции выполняются параллельно и не могут оказывать влияния друг на друга
- **Durability (надежность)** — обеспечивает сохранность всех данных после фиксации успешно выполненной транзакции, независимо от возможных сбоев системы

Типы транзакций

- 1) **Явные** - явное указание:
 - начала транзакции (BEGIN TRAN или BEGIN TRANSACTION),
 - ее фиксации в случае успешного завершения (COMMIT TRAN или COMMIT TRANSACTION)
 - с возможностью прерывания (отката) транзакции при возникновении определенного условия (ROLLBACK TRAN или ROLLBACK TRANSACTION)
- 2) **Неявные** - транзакции, для которых начало и конец не указываются явным образом. При данных операциях:
 - ALTER TABLE, CREATE, DROP, SELECT, INSERT, DELETE, UPDATE, GRANT, REVOKE, OPEN, FETCH, TRUNCATE TABLE

3) **Автоматические** - каждая успешно выполненная операция фиксируется, в противном случае откатывается

→ Для того чтобы перейти в этот режим необходимо выполнить команду SET IMPLICIT_TRANSACTION с параметром OFF, для выхода SET IMPLICIT_TRANSACTION ON

Блокировки, применяемые при выполнении транзакций, препятствуют взаимодействию с объектами другим процессам, тем самым могут служить причиной взаимной изоляции транзакций.

Уровни изолированности транзакций:

1. **READ UNCOMMITTED** — позволяет считывать данные, которые были изменены, но не были зафиксированы (*не предотвращает ни одно из возможных нарушений*)
2. **READ COMMITTED** — не позволяет считывать данные, которые были изменены, но не были зафиксированы (используется по умолчанию) (*предотвращает чтение незафиксированных данных*)
3. **REPEATABLE READ** — никакая транзакция не может изменять данные, считанные текущей транзакцией до ее завершения (*предотвращает чтение незафиксированных данных и неповторяемое чтение*)
4. **SERIALIZABLE** — любые другие транзакции не могут вставлять/изменять/удалять данные, которые попадают в диапазон записей, удовлетворяющих условию, которое задано в операторе WHERE текущей транзакции (*предотвращает чтение незафиксированных данных, неповторяемое чтение и фантомы*)

Домашняя работа 1

В базу данных Hospital добавить таблицу Patients со следующими столбцами:

1) Id

- a) Тип данных - int
- b) Авто приращение.
- c) Не может содержать null-значения.
- d) Первичный ключ.

2) Department

- a) Тип данных - nvarchar(100)
- b) Не может содержать null-значения.
- c) Не может быть пустым

3) StartDate

- a) Тип данных - date
- b) Не может содержать null-значения.

4) EndDate

- a) Тип данных - date
- b) Не может содержать null-значения.
- c) Не может быть раньше StartDate

5) BirthDate

- o Тип данных - date
- o Не может содержать null-значения
- o Не может быть больше текущей даты

6) Surname

- o Тип данных - nvarchar(max)
- o Не может содержать null-значения
- o Не может быть пустым

7) Doctor

- o Тип данных - nvarchar(max)
- o Не может содержать null-значения
- o Не может быть пустым
- o Значение по умолчанию - Doe

8) Disease

- o Тип данных - nvarchar(100)
- o Не может содержать null-значения.
- o Не может быть пустым
- o Значение по умолчанию - Undefined

Домашняя работа 2

Написать запросы к базе данных Hospital:

- 1) Вывести информацию обо всех пациентах, находящихся в больнице;
- 2) Показать данные о пациентах, которые лежат в определенном отделении;
- 3) Получить данные о пациентах, которые лежат в больнице больше месяца, отсортировав их по возрастанию даты поступления;
- 4) Вывести информацию о пациентах, которые были выписаны в прошлом месяце (вам поможет стандартная функция GETDATE() языка T-SQL, которая позволяет получить текущую дату);
- 5) Показать информацию о пациентах, которые лежали в больнице с октября по декабрь прошлого года в определенном отделении;
- 6) Вывести информацию о самом молодом пациенте и его возраст (для написания этого запроса вам следует самостоятельно изучить стандартную функцию DATEDIFF(), информацию о которой вы можете получить по следующей ссылке <https://docs.microsoft.com/en-us/sql/t-sql/functions/datediff-transact-sql>);
- 7) Показать всех пациентов, фамилия которых начинается на букву Р;
- 8) Вывести данные о пациентах, которых лечит определенный врач с одинаковыми заболеваниями;
- 9) Удалить всех пациентов, которые были выписаны больше чем полгода назад